claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-a30dddadefba9b41a.jsonl`
- SHA-256: `22ea78c77a49f3323a84c627a2d2e072f822384b5e34c8ba62987c950cad859d`
- Source modified: `2026-06-18T01:42:13+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:40:32.162Z)

```
Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read
READ-ONLY in this worktree (origin/master), DO NOT modify:
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch
what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE
trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the
trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list
is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py,
backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

CANDIDATE:
{
  "title": "run_process_and_stitch.py: len()/truthiness on un-unwrapped Result breaks under the
motion segmenter",
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-
base/scripts/run_process_and_stitch.py",
  "line": "81-84",
  "trigger": "Set config `indexing.default_segmenter` to 'motion' (config.json currently ships
'gemini'; code default 'yolo_hybrid'), then run the script on a non-cached video so the segment
branch (line 81) executes.",
  "why_wrong": "Line 81 `segments = segmenter.process_video(...)` returns a `Result`
(Success/Failure) for the motion segmenter. Line 82 `len(segments)` then raises `TypeError`
(Success has no `__len__`), and line 84 `if not segments:` would also be wrong because a
`Success`/`Failure` dataclass instance is always truthy, so the \"No play segments detected\"
guard can never fire. The result is never unwrapped (no `.value` access, no `isinstance(_,
Failure)` check), unlike backend/main.py. Latent for the shipped default (gemini returns a bare
list) but a real defect for the motion segmenter.",
  "severity": "low",
  "proposed_fix": "Unwrap before use, e.g. `result = segmenter.process_video(video_id,
video_path); if isinstance(result, Failure): print(f\"[FAIL] {result.message}\"); return;
segments = result.value if hasattr(result, 'value') else result` ? then `len(segments)` and `if
not segments:` operate on the list.",
  "confidence": 0.7,
  "klass": "cli_fastapi_request_handling"
}

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-
base/C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-
base/scripts/run_process_and_stitch.py and related code/tests READ-ONLY before judging.

LENS ? REACHABILITY: prove a real path reaches it, or set real=false.
```

2. assistant (2026-06-18T01:40:34.661Z)

I'll investigate this candidate bug. Let me first read the target file and understand the code paths.

3. user (2026-06-18T01:40:35.140Z)

```
1       import os
2       import json
3       import logging
4       from dotenv import load_dotenv
5
6       load_dotenv()
7
8       # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
9       # Quiet third-party HTTP chatter by default; re-honoured against the config knob
10      # (logging.verbose_http) once the config is loaded in main().
11      from backend.utils.logging_setup import setup_logging
12      setup_logging()
13
14      from backend.infrastructure.database import Database
15      from backend.config import load_config
16      from backend.utils.hashing import compute_video_id
17      from backend.pipeline.segmenters import segmenter_registry
18      from backend.pipeline.stitcher.compiler import VideoCompiler
19      from backend.tools.export import format_rally_summary
20
21      logger = logging.getLogger(__name__)
22
23      def main():
24          video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
25          output_dir = r"C:\Users\avidu\Videos\Testing"
26          output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
27
28          print("Loading config...")
29          cfg = load_config()  # typed AppConfig ? single source of defaults
30          # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
31          setup_logging(verbose_http=cfg.logging.verbose_http)
32
33          print("Verifying files exist...")
34          if not os.path.exists(video_path):
35              print(f"Error: Video file not found at {video_path}")
36              return
37
38          os.makedirs(cfg.output.output_dir, exist_ok=True)
39          db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
40          db = Database(db_path)
41
42          # Calculate MD5 of the video to uniquely identify it
43          import time
44          print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
45          t0 = time.time()
46          video_id = compute_video_id(video_path)
47          print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
48
49          segments = []
50          cached_video = db.get_video(video_id)
51          cache_hit = False
52
53          if cached_video and cached_video.get("status") == "processed":
54              segments = db.query_play_segments(video_id, {})
55              if len(segments) > 0:
56                  print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
57                  choice = input("Would you like to (1) Run stitching on the cached segments,
```

```
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
                if choice == '1':
                    print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
                    cache_hit = True
                else:
                    print("[CACHE] Ignoring cache...", flush=True)
            else:
                print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)

        if not cache_hit:
            print("[CACHE] Initializing clean indexing...", flush=True)
            db.clear_video_data(video_id)

            print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
            db.add_video(video_id, video_path, "processed", [])

            print("Running segmenter & indexing...", flush=True)
            seg_name = cfg.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return
            print(f"Using segmenter: {seg_name}", flush=True)
            segmenter = segmenter_cls(db, cfg)
            segments = segmenter.process_video(video_id, video_path)
            print(f"Successfully indexed {len(segments)} play segments.", flush=True)

        if not segments:
            print("No play segments detected. Cannot compile highlights.")
            return


        end_padding = cfg.stitching.end_padding
        begin_padding = cfg.stitching.begin_padding
        min_shot_count = cfg.stitching.min_shot_count

        if cache_hit:
            print("\n--- Stitching Customizations ---")
            try:
                bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
                if bpad_input:
                    begin_padding = float(bpad_input)

                pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
                if pad_input:
                    end_padding = float(pad_input)

                shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
                if shot_input:
                    min_shot_count = int(shot_input)
            except ValueError:
                print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")

        print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)

        # Filter segments where shot_count >= min_shot_count
        time_pairs = []
```

```
        skipped_count = 0
        for s in segments:
            meta = s.get("metadata", {})
            if isinstance(meta, str):
                try:
                    meta = json.loads(meta)
                except Exception:
                    meta = {}
            shot_count = meta.get("shot_count", 0)
            if shot_count >= min_shot_count:
                time_pairs.append((s["start_time"], s["end_time"]))
            else:
                skipped_count += 1

        if skipped_count > 0:
            print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
        print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)

        # End-of-run rally table for manual QA ? sourced from the produced play_segments so
        # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
        # aligned rows) so it survives even if the table is the last thing the owner scans.
        def _log_rally_summary() -> None:
            logger.info("\n%s", format_rally_summary(segments))

        if not time_pairs:
            print("No play segments passed the filtering criteria. Cannot compile
highlights.")
            _log_rally_summary()
            return

        compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding,
                                 watermark=cfg.stitching.watermark)
        try:
            out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
            print(f"Stitched highlight video saved to: {out_path}", flush=True)
        except Exception as e:
            print(f"Error during stitching: {e}", flush=True)

        _log_rally_summary()

    if __name__ == "__main__":
        main()

```

4. assistant (2026-06-18T01:40:37.592Z)

Now let me examine the segmenter interface to determine what `process_video` actually returns. The candidate's claim hinges on the motion segmenter returning a `Result` rather than a list.

5. user (2026-06-18T01:40:38.399Z)

```
      from abc import ABC, abstractmethod
      from typing import List, Dict, Any, Type, Optional
      from backend.infrastructure.database import Database
      from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)

      class BasePlaySegmenter(ABC):
          def __init__(self, db: Database, config: Dict[str, Any]):
              self.db = db
              self.config = config

```

```
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

6. assistant (2026-06-18T01:40:41.716Z)

This confirms the contract: `process_video` returns a `Result[List[Dict[str, Any]]]`. The base class docstring explicitly says to use `isinstance(result, Failure)` to distinguish outcomes. Now let me check the actual segmenter implementations to verify what each returns, and how `backend/main.py` handles it.

7. user (2026-06-18T01:40:42.479Z)

```
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
```

8. user (2026-06-18T01:40:43.031Z)

```
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
14
15      This module has no dependencies on the rest of the app.
16      """
17
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Generic, TypeVar, List, Optional, Any, Union
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
```

```
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
61
```

9. assistant (2026-06-18T01:40:46.432Z)

The `Result` type is confirmed: `Success`/`Failure` are dataclasses with no `__len__` and are always truthy. Now let me verify what each segmenter actually returns, especially `motion.py` and `gemini.py`, and how `backend/main.py` correctly unwraps the result.

10. user (2026-06-18T01:40:47.748Z)

```
1       import cv2
2       import time
3       import random
4       from typing import List, Dict, Any, Tuple
5       import logging
6       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7       from backend.results import Success, Failure  # standardized contract
8
9       logger = logging.getLogger(__name__)
10
11      class MotionPlaySegmenter(BasePlaySegmenter):
12          @property
13          def name(self) -> str:
14              return "motion"
15
16          @property
17          def description(self) -> str:
18              return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21              """
22              Processes a video to detect play segments using motion heuristics,
23              populates SQLite with detected segments, and indexes detailed events.
24              """
25              if not player_pool:
26                  player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28              cap = cv2.VideoCapture(video_path)
29              if not cap.isOpened():
30                  logger.error(f"Segmenter could not open video: {video_path}")
31                  return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33              fps = cap.get(cv2.CAP_PROP_FPS)
34              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
35              duration = total_frames / fps if fps > 0 else 0
36
37              # Sub-sample settings (5 FPS for analysis)
38              analysis_fps = 5.0
39              frame_interval = max(1, int(fps / analysis_fps))
40
41              idx_cfg = self.config.get("indexing", {})
42              motion_threshold = idx_cfg.get("motion_threshold", 0.08)
43              min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
44              dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
```

```
45
46              prev_gray = None
47              motion_signals = []
48              timestamps = []
49
50              frame_idx = 0
51              processed_count = 0
52              t_start = time.time()
53              # Emit a progress line roughly every 5% of the video so long runs are
54              # observable instead of silent (a full 1080p match is ~tens of minutes).
55              progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
56              next_progress = progress_step
57              logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
58                          f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
59              while True:
60                  # Skip decoding for intermediate frames using cap.grab() for high
performance
61                  for _ in range(frame_interval - 1):
62                      if not cap.grab():
63                          break
64                      frame_idx += 1
65
66                  ret, frame = cap.read()
67                  if not ret:
68                      break
69
70                  # Process frame
71                  gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
72                  # Apply Gaussian Blur to smooth noise
73                  gray = cv2.GaussianBlur(gray, (21, 21), 0)
74
75                  t = frame_idx / fps
76                  timestamps.append(t)
77
78                  if prev_gray is not None:
79                      # Frame absolute difference (highly optimized motion tracking)
80                      diff = cv2.absdiff(prev_gray, gray)
81                      _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
82
83                      # Dilate to fill gaps
84                      thresh = cv2.dilate(thresh, None, iterations=2)
85
86                      # Motion ratio
87                      motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
88                      motion_signals.append(motion_ratio)
89                  else:
90                      motion_signals.append(0.0)
91
92                  prev_gray = gray
93                  frame_idx += 1
94                  processed_count += 1
95
96                  if progress_step and frame_idx >= next_progress:
97                      pct = 100.0 * frame_idx / total_frames
98                      elapsed = time.time() - t_start
99                      eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
100                     logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
101                                 f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
102                     next_progress += progress_step
103
104                 if max_frames is not None and processed_count >= max_frames:
```

```
                    logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
                    break

                if frame_idx >= total_frames:
                    break

            cap.release()

            # Smooth signal using a moving average
            window_size = 5
            smoothed = []
            for i in range(len(motion_signals)):
                start = max(0, i - window_size // 2)
                end = min(len(motion_signals), i + window_size // 2 + 1)
                smoothed.append(sum(motion_signals[start:end]) / (end - start))

            # Detect active play boundaries
            in_segment = False
            segment_start = None
            raw_segments: List[Tuple[float, float]] = []

            for i, t in enumerate(timestamps):
                signal = smoothed[i]
                if signal > motion_threshold:
                    if not in_segment:
                        in_segment = True
                        segment_start = t
                else:
                    if in_segment:
                        in_segment = False
                        raw_segments.append((segment_start, t))

            # Handle video ending while in active segment
            if in_segment:
                raw_segments.append((segment_start, timestamps[-1]))

            # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
            merged_segments: List[Tuple[float, float]] = []
            if raw_segments:
                curr_start, curr_end = raw_segments[0]
                for next_start, next_end in raw_segments[1:]:
                    if next_start - curr_end < dead_time_merge_gap:
                        # Merge
                        curr_end = next_end
                    else:
                        merged_segments.append((curr_start, curr_end))
                        curr_start, curr_end = next_start, next_end
                merged_segments.append((curr_start, curr_end))

            # Post-Processing: 2. Filter segments shorter than min_segment_duration
            final_segments: List[Tuple[float, float]] = [
                r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
            ]

            # Populate SQLite DB with segments and metadata events
            segments_data = []
            for start, end in final_segments:
                # Assign Server and Receiver randomly from pool
                server, receiver = random.sample(player_pool, 2)

                # Formulate dynamic rich tags
                dur = end - start
                metadata = {
                    "shot_count": random.randint(4, 25),
```

```
169                     "intensity": "high" if dur > 12 else "medium",
170                     "avg_speed_kmh": round(random.uniform(40, 85), 1)
171                 }
172 
173                 segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
174 
175                 if segment_id:
176                     # Add sub-segment events
177                     # A: Serve Event (always at start)
178                     serve_foul = random.random() < 0.15 # 15% chance of serve foul
179                     self.db.add_event(
180                         segment_id=segment_id,
181                         timestamp=start + 0.5,
182                         event_type="serve",
183                         player=server,
184                         details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
185                     )
186 
187                     # B: Smash Events (1 to 3 random smashes per segment)
188                     smashes_count = random.randint(0, 3)
189                     for _ in range(smashes_count):
190                         smash_time = random.uniform(start + 1.5, end - 1.0)
191                         smasher = random.choice([server, receiver])
192                         self.db.add_event(
193                             segment_id=segment_id,
194                             timestamp=round(smash_time, 2),
195                             event_type="smash",
196                             player=smasher,
197                             details={"speed_kmh": round(random.uniform(120, 250), 1)}
198                         )
199 
200                     # C: Segment Termination Event (foul/winner)
201                     foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
202                     is_foul = random.random() > 0.4
203                     error_player = random.choice([server, receiver])
204 
205                     self.db.add_event(
206                         segment_id=segment_id,
207                         timestamp=end - 0.2,
208                         event_type="foul" if is_foul else "winner",
209                         player=error_player,
210                         details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
211                     )
212 
213                     # Store segment representation to return to caller
214                     segments_data.append({
215                         "id": segment_id,
216                         "start_time": start,
217                         "end_time": end,
218                         "duration": dur,
219                         "server": server,
220                         "receiver": receiver,
221                         "metadata": metadata
222                     })
223 
224             return Success(value=segments_data)
225 
226     # Register the segmenter
227     segmenter_registry.register(MotionPlaySegmenter)
228 
```

11. assistant (2026-06-18T01:40:51.709Z)

Confirmed for the motion segmenter: it returns `Failure(...)` on line 31 and `Success(value=segments_data)` on line 224. Both are dataclass instances with no `__len__`. So `len(segments)` on line 82 of the script would raise `TypeError`, and `if not segments:` on line 84 would always be False.

Now let me check `gemini.py` (the shipped default per config.json) to confirm whether it also returns a `Result` or a bare list ? and check `backend/main.py` to see the correct unwrapping pattern.

12. user (2026-06-18T01:40:52.753Z)

```
1       import os
2       import time
3       import logging
4       import subprocess
5       import concurrent.futures
6       from typing import List, Dict, Any, Tuple
7       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8       from backend.utils.video import get_video_duration, extract_video_chunk
9       from backend.sports import sport_registry
10      from backend.providers import provider_registry
11      from backend.reporting import run_signals
12
13      logger = logging.getLogger(__name__)
14
15
16      def _chunk_workdir(video_id: str) -> str:
17          """Per-video chunk staging dir under output/chunks/<video_id>.
18
19          Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20          otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21          in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22          """
23          return os.path.join("output", "chunks", video_id)
24
25
26      class GeminiPlaySegmenter(BasePlaySegmenter):
27          @property
28          def name(self) -> str:
29              return "gemini"
30
31          @property
32          def description(self) -> str:
33              return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
34
35          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36              """
37              Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38              populates SQLite, and indexes detailed events.
39              """
40              # Get sport profile config
41              sport_name = self.config.get("sport", "badminton")
42              try:
43                  sport_profile = sport_registry.get(sport_name)
44              except KeyError as e:
45                  logger.error(str(e))
46                  return []
47
48              # Get AI provider and model config
49              idx_cfg = self.config.get("indexing", {})
50              provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
```

```
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))

            # Get appropriate API key based on the provider
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} segmenter, set your API key. "
                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                )
                return []

            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            # A: Check for debug_duration to trim the video first
            debug_duration = self.config.get("indexing", {}).get("debug_duration")
            video_to_upload = video_path
            is_trimmed = False
            temp_trimmed_path = None

            if debug_duration:
                logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
                temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
                os.makedirs("output", exist_ok=True)
                if os.path.exists(temp_trimmed_path):
                    try:
                        os.remove(temp_trimmed_path)
                    except Exception:
                        pass

                cmd = [
                    "ffmpeg", "-y",
                    "-ss", "0",
                    "-t", str(debug_duration),
                    "-i", video_path,
                    "-c", "copy",
                    temp_trimmed_path
                ]
                logger.debug(f"Running command: {' '.join(cmd)}")
                try:
                    subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
                    logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
                    video_to_upload = temp_trimmed_path
                    is_trimmed = True
                except Exception as e:
                    logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")


            # Helper to clean up local temp files
            def cleanup_temp_files():
                if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
                    try:
                        os.remove(temp_trimmed_path)
```

```
                    except Exception:
                        pass

            # 1. Determine actual video duration
            video_duration = get_video_duration(video_to_upload)
            if video_duration > 0:
                logger.info(f"Detected video duration: {video_duration:.1f}s")
            else:
                logger.warning("Could not detect video duration. Prompt will not include
duration context.")

            # 2. Decide whether to use chunked processing
            idx_cfg = self.config.get("indexing", {})
            chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
            chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
            use_chunking = video_duration > 0 and video_duration > chunk_duration
            # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
            # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
            # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
            ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)

            ai_segments = []
            # Per-video chunk workdir (concurrency isolation): a concurrent run for another
            # video keeps its own dir, so cleanup here never removes another run's chunks.
            chunks_dir = _chunk_workdir(video_id)

            if use_chunking:
                # --- CHUNKED PROCESSING ---
                step = chunk_duration - chunk_overlap
                chunk_starts = []
                t = 0.0
                while t < video_duration:
                    chunk_starts.append(t)
                    t += step
                num_chunks = len(chunk_starts)
                logger.info(
                    f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
                    f"({chunk_duration}s each, {chunk_overlap}s overlap)."
                )

                os.makedirs(chunks_dir, exist_ok=True)

                def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_chunk_dur = chunk_end - chunk_start
                    chunk_label = f"Chunk {ci+1}/{num_chunks}"
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    # Extract chunk with ffmpeg
                    logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
                    success = extract_video_chunk(
                        video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
                        reencode=ai_scale_width > 0,
                        scale_width=ai_scale_width if ai_scale_width > 0 else None,
                    )
                    if not success:
                        logger.error(f"Failed to extract {chunk_label}. Skipping.")
                        return chunk_label, [], {}

                    # Measure chunk size
                    try:
```

```
                        chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
                    except Exception:
                        chunk_size_mb = 0.0

                    # Get prompt from sport profile
                    prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)

                    # Process this chunk through the provider
                    chunk_segments, metrics = provider.analyze_video(
                        chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
                    )
                    metrics["file_size_mb"] = chunk_size_mb
                    metrics["segments_found"] = len(chunk_segments)

                    # Offset timestamps back to full-video coordinates
                    for segment in chunk_segments:
                        if "start_time" in segment:
                            try:
                                segment["start_time"] = float(segment["start_time"]) +
chunk_start
                            except (ValueError, TypeError):
                                pass
                        if "end_time" in segment:
                            try:
                                segment["end_time"] = float(segment["end_time"]) +
chunk_start
                            except (ValueError, TypeError):
                                pass

                    logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")

                    # Clean up chunk file
                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                    return chunk_label, chunk_segments, metrics

                # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
                # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
                # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
                # misleading "credits depleted" 429 under 5 simultaneous uploads.
                all_metrics = {}
                max_workers = max(1, int(self.config.get("indexing",
{}).get("ai_max_workers", 5)))
                logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
                t_chunking_start = time.time()
                with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
                    futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
                    for future in concurrent.futures.as_completed(futures):
                        label, segments, metrics = future.result()
                        ai_segments.extend(segments)
                        if metrics:
                            all_metrics[label] = metrics

                # Oversize refusals must be LOUD: each one is a time range with NO analysis,
                # so "0 segments / success" here would be a silent failure, not "no
```

```
rallies".
                oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
                if oversize_skips:
                    logger.error(
                        f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
                        f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
                        f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
                        f"downscaled before upload."
                    )
                    run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)

                # Print Summary Log
                summary_lines = [
                    "",
                    "=" * 60,
                    "               GEMINI CHUNKING SUMMARY",
                    "=" * 60,
                    f" Total time: {time.time() - t_chunking_start:.1f}s",
                    f" Total chunks: {num_chunks}",
                    f" Total play segments found: {len(ai_segments)}",
                    "-" * 60,
                    f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
                    "-" * 60,
                ]
                for i in range(num_chunks):
                    lbl = f"Chunk {i+1}/{num_chunks}"
                    m = all_metrics.get(lbl, {})
                    sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
                    upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
                    prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
                    gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
                    seg = m.get('segments_found', 0) if m else 0
                    summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
                summary_lines.append("=" * 60)
                logger.info("\n".join(summary_lines))

                # Clean up local temp files
                cleanup_temp_files()
                try:
                    os.rmdir(chunks_dir)
                except Exception:
                    pass

                logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")

            else:
                # --- SINGLE-FILE PROCESSING (short videos) ---
                try:
                    file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
                except Exception:
                    file_sz = 0.0

                # Get prompt from sport profile
                prompt = sport_profile.get_prompt(chunk_duration=video_duration)
                ai_segments, metrics = provider.analyze_video(
                    video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
                )
```

```
                if metrics.get("oversize_skipped"):
                    logger.error(
                        f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
                        f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
                    )
                    run_signals.incr(video_id, "oversize_clips_skipped", 1)
                summary_lines = [
                    "",
                    "=" * 50,
                    "        GEMINI SINGLE FILE SUMMARY",
                    "=" * 50,
                    f" File Size: {file_sz:.1f} MB",
                    f" Upload time: {metrics.get('upload_time', 0):.1f}s",
                    f" API Processing time: {metrics.get('process_time', 0):.1f}s",
                    f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
                    f" Play segments found: {len(ai_segments)}",
                    "=" * 50,
                ]
                logger.info("\n".join(summary_lines))
                cleanup_temp_files()

            if not ai_segments:
                return []

            # Populate SQLite DB with parsed play segments
            segments_data = []
            logger.info(f"Processing {len(ai_segments)} play segments through
validation...")

            def parse_time(val) -> float:
                """Converts a timestamp value to total float seconds.
                Handles float/int pass-through, plain numeric strings, and
                colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
                when unexpected formats are encountered."""
                if isinstance(val, (int, float)):
                    return float(val)
                if isinstance(val, str):
                    val = val.strip()
                    if ":" in val:
                        # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
                        # Convert it, but warn so the issue is visible in logs.
                        parts = val.split(":")
                        parts.reverse()
                        total = 0.0
                        for i, p in enumerate(parts):
                            try:
                                total += float(p) * (60 ** i)
                            except ValueError:
                                pass
                        logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
                        return total
                    try:
                        return float(val)
                    except ValueError:
                        logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
                        return 0.0
                logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
                return 0.0

```

```
            # 7. Parse timestamps and build candidate segment list
            idx_cfg = self.config.get("indexing", {})
            min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
            max_segment_duration = 90.0  # Singles play segments almost never exceed this

            candidates = []
            for idx, segment in enumerate(ai_segments):
                start = parse_time(segment.get("start_time", 0.0))
                end = parse_time(segment.get("end_time", 0.0))
                if start >= end:
                    logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
                    continue
                candidates.append({
                    "idx": idx,
                    "start": start,
                    "end": end,
                    "raw": segment
                })

            # 8. Post-hoc validation pass
            logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
            validated = []
            rejected = []

            for c in candidates:
                label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"

                # A: Reject segments that start beyond the video
                if video_duration > 0 and c["start"] >= video_duration:
                    rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
                    continue

                # B: Clamp end_time to video duration
                if video_duration > 0 and c["end"] > video_duration:
                    logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
                    c["end"] = video_duration

                dur = c["end"] - c["start"]

                # C: Reject segments shorter than min_segment_duration
                if dur < min_segment_duration:
                    rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
                    continue

                # D: Flag suspiciously long segments
                if dur > max_segment_duration:
                    logger.warning(
                        f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
                        f"This may be a detection error. Keeping but flagging."
                    )

                validated.append(c)

            # E: Resolve overlapping segments (keep the longer one)
            if len(validated) > 1:
                # Sort by start time
                validated.sort(key=lambda r: r["start"])
                non_overlapping = [validated[0]]
                for curr in validated[1:]:
```

```
                    prev = non_overlapping[-1]
                    # Check if current segment overlaps with the previous one
                    if curr["start"] < prev["end"]:
                        prev_dur = prev["end"] - prev["start"]
                        curr_dur = curr["end"] - curr["start"]
                        kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
                        kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
                        dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
                        rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
                        if kept == curr:
                            non_overlapping[-1] = curr  # Replace previous with current
                    else:
                        non_overlapping.append(curr)
                validated = non_overlapping

            # Log validation summary
            if rejected:
                reject_lines = [f"  ? {label}: {reason}" for label, reason in rejected]
                logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
            logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")

            stitch_cfg = self.config.get("stitching", {})
            begin_pad = stitch_cfg.get("begin_padding", 0.5)
            end_pad = stitch_cfg.get("end_padding", 0.5)

            total_highlight_dur = 0.0
            for c in validated:
                total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad

            mins = int(total_highlight_dur // 60)
            secs = int(total_highlight_dur % 60)
            logger.info(
                f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
                f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
            )

            # 9. Populate SQLite DB with validated play segments
            for c in validated:
                start = c["start"]
                end = c["end"]
                segment = c["raw"]
                dur = end - start

                # Extract shot_count from Gemini or compute estimate if not present
                shot_count = segment.get("shuttle_exchange_count")
                if shot_count is None:
                    shot_count = max(3, int(dur / 0.8))  # 1 shot every 0.8 seconds as
fallback estimate
                else:
                    try:
                        shot_count = int(shot_count)
                    except Exception:
                        shot_count = max(3, int(dur / 0.8))

                # Only store metrics we actually know ? no fabricated data
                metadata = {
                    "shot_count": shot_count,
                    "shot_count_confidence": segment.get("shot_count_confidence",
```

```
"Unknown"),
455                     "detector": model_name
456                 }
457
458                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
459
460                 if segment_id:
461                     segments_data.append({
462                         "id": segment_id,
463                         "start_time": start,
464                         "end_time": end,
465                         "duration": dur,
466                         "metadata": metadata
467                     })
468
469             return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473
```

13. assistant (2026-06-18T01:40:57.207Z)

Critical finding: the gemini segmenter returns **bare lists** (`return []` on lines 46, 62, 68, 303 and `return segments_data` on line 469), NOT a `Result`. So gemini does NOT raise the TypeError. This confirms the candidate's "latent for the shipped default" claim.

But the motion segmenter returns `Success`/`Failure`. Let me verify the other segmenters and check `backend/main.py` for the correct unwrapping pattern, plus the config defaults.

14. user (2026-06-18T01:40:59.779Z)

```
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py:46:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py:62:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py:68:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py:303:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py:469:        return
segments_data
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:119:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:144:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:149:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:154:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:189:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:200:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:247:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:283:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\trajectory_hybrid.py:337:        return
segments_data
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py:66:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py:82:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py:88:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py:95:            return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py:288:
return []
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py:366:        return
segments_data
```

```
.claude\worktrees\hunt-base\backend\pipeline\segmenters\motion.py:31:            return
Failure(message=f"Could not open video: {video_path}", is_recoverable=False)
.claude\worktrees\hunt-base\backend\pipeline\segmenters\motion.py:224:        return
Success(value=segments_data)
```

15. user (2026-06-18T01:41:00.778Z)

```
192:            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
201:        if isinstance(result, Failure):
213:        segments = result.value if hasattr(result, "value") else result
347:def process_video(req: ProcessRequest):
835:            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
838:            if isinstance(result, Failure):
843:                segments = result.value if hasattr(result, "value") else result
```

16. assistant (2026-06-18T01:41:05.754Z)

This is very revealing. Among ALL the segmenters, **only `motion.py` returns a `Result`** (`Success`/`Failure`). The gemini, trajectory_hybrid, yolo_hybrid, wasb_hybrid, tracknet_hybrid, and fusion_hybrid segmenters all return bare lists. So the inconsistency is real but localized.

`backend/main.py` handles BOTH cases defensively (lines 201/213 and 838/843): it checks `isinstance(result, Failure)` first, then unwraps with `result.value if hasattr(result, "value") else result`. This is the correct pattern that `run_process_and_stitch.py` lacks.

Now the key question for reachability: can `motion` actually be selected as `default_segmenter`? Let me check the config default and config.json.

17. user (2026-06-18T01:41:07.700Z)


```
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-flash-latest",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [
                35,
                40,
                40
            ],
            "court_green_upper": [
                85,
                255,
                255
            ],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
        "default_segmenter": "gemini",
        "use_gpu": true,
        "motion_threshold": 0.08,
        "min_segment_duration": 3.0,
        "dead_time_merge_gap": 2.0,
        "chunk_duration": 90,
        "chunk_overlap": 10,
```

```
        "detector_model": "fasterrcnn_resnet50_fpn",
        "detector_score_threshold": 0.5,
        "yolo_velocity_threshold": 0.15,
        "yolo_frame_skip": 3,
        "yolo_tracker": "bytetrack.yaml",
        "yolo_prefetch_workers": 2,
        "min_action_window_duration": 2.0,
        "log_yolo_candidates": true,
        "store_yolo_candidates": true,
        "ai_handoff_padding": 2.0,
        "ai_max_workers": 1,
        "tracknet": {
            "wsl_distro": "Ubuntu",
            "repo_dir": "~/models/TrackNetV4",
            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
            "conda_env": "TrackNetV4",
            "weights_path": "",
            "queue_length": 5,
            "stage_in_wsl": true,
            "wsl_stage_dir": "~/clips",
            "timeout_sec": 1800,
            "velocity_threshold": 0.02
        },
        "wasb": {
            "wsl_distro": "Ubuntu",
            "repo_dir": "~/models/WASB-SBDT",
            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
            "conda_env": "wasb",
---CONFIG DEFAULTS---
backend/config/models.py:118:    default_segmenter: str = "yolo_hybrid"
backend/config/models.py:192:    @field_validator("default_segmenter")
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
```

18. assistant (2026-06-18T01:41:10.587Z)

So the shipped `config.json` has `default_segmenter: "gemini"` (bare list ? no crash), and the config default is `"yolo_hybrid"` (also bare list ? no crash). To trigger the bug, a user must explicitly set `default_segmenter: "motion"`. Let me check the field validator to confirm `motion` is a valid, accepted value.

19. user (2026-06-18T01:41:12.045Z)

```
185         # Optional debug knob: trim every video to the first N seconds before processing.
186         debug_duration: Optional[float] = None
187
188         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
189         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
190         fusion: FusionConfig = Field(default_factory=FusionConfig)
191
192         @field_validator("default_segmenter")
193         @classmethod
194         def segmenter_must_be_registered(cls, v: str) -> str:
195             # Registry check happens at runtime in main/segmenter selection.
196             # Here we just allow any string for forward compatibility.
197             return v
198
199         @model_validator(mode="after")
200         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
201             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
202             # step makes `while t < duration: t += step` loop forever, growing memory before
any
203             # GPU work. Reject the bad config at load time instead.
204             if self.chunk_overlap >= self.chunk_duration:
205                 raise ValueError(
```

```
206                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
207                     f"({self.chunk_duration}); otherwise chunking never advances."
208                 )
209             return self
210
211
212     class OutputConfig(BaseModel):
213         default_highlights_name: str = "highlights.mp4"
214         db_name: str = "sports_indexer.db"
```

20. assistant (2026-06-18T01:41:18.552Z)

The validator allows any string for forward compatibility, so `"motion"` is a fully valid config value, and `motion` is registered in the registry (`segmenter_registry.register(MotionPlaySegmenter)`). The script's `segmenter_registry.get("motion")` would succeed and instantiate the motion segmenter.

Now let me confirm the trigger path concretely. With `default_segmenter: "motion"` and a non-cached video:
- Line 75: `segmenter_cls = segmenter_registry.get("motion")` ? returns `MotionPlaySegmenter`
- Line 81: `segments = segmenter.process_video(video_id, video_path)` ? returns `Success(value=[...])` or `Failure(...)`
- Line 82: `print(f"Successfully indexed {len(segments)} play segments.")` ? **`len(Success)` raises `TypeError: object of type 'Success' has no len()`**

Let me verify `Success`/`Failure` genuinely have no `__len__` and are truthy, and confirm the script's hardcoded path requires `cv2` (which gates real-world reachability). Let me also double check there's no `__len__`/`__bool__` defined anywhere on these dataclasses.

21. user (2026-06-18T01:41:22.528Z)

```
Success truthy: True
Failure truthy: True
has __len__ Success: False
has __len__ Failure: False
len(Success) -> TypeError object of type 'Success' has no len()
len(Failure) -> TypeError object of type 'Failure' has no len()
if not segments -> would skip guard? True
```

22. assistant (2026-06-18T01:41:27.722Z)

Empirically confirmed:
- `len(Success(...))` raises `TypeError: object of type 'Success' has no len()` ? line 82 crashes.
- `Success`/`Failure` are always truthy ? line 84 `if not segments:` can never fire (the "No play segments detected" guard is defeated).

Now let me verify reachability isn't blocked by something else. The script imports `backend.pipeline.segmenters` (line 17), which imports motion.py, which does `import cv2` at module top. This environment lacks cv2, so I can't run the script E2E here ? but that's an environment limitation, not a code path block. On the owner's GPU/WSL machine cv2 is present. Let me confirm the script actually imports the motion module (via the registry `__init__`) so the registry contains `motion`.

23. user (2026-06-18T01:41:28.538Z)

```
1       from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2       from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3       from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4       from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6       from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7       from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
```

```
8
9       __all__ = [
10          "segmenter_registry",
11          "BasePlaySegmenter",
12          "MotionPlaySegmenter",
13          "GeminiPlaySegmenter",
14          "HybridYoloSegmenter",
15          "TrackNetHybridSegmenter",
16          "WasbHybridSegmenter",
17          "FusionSegmenter",
18      ]
19
```

24. assistant (2026-06-18T01:41:32.231Z)

The `motion` segmenter is registered when `backend.pipeline.segmenters` is imported (line 17 of the script imports `segmenter_registry`, which triggers `__init__.py`). So `segmenter_registry.get("motion")` returns the class.

Let me do a final reachability sanity check on whether there's anything that would block `motion` selection in this CLI script ? and confirm the script is a real, used entrypoint (not dead/abandoned). Let me check if it's referenced anywhere and whether the validation flow would reject motion.

25. user (2026-06-18T01:41:33.443Z)

Found 9 files
.claude\worktrees\hunt-base\tests\test_api_endpoints.py
.claude\worktrees\hunt-base\scripts\README.md
.claude\worktrees\hunt-base\run_eval.py
.claude\worktrees\hunt-base\ruff.toml
.claude\worktrees\hunt-base\docs\EVALUATION.md
.claude\worktrees\hunt-base\docs\CODE_MAP.md
.claude\worktrees\hunt-base\backend\utils\hashing.py
.claude\worktrees\hunt-base\backend\main.py
.claude\worktrees\hunt-base\README.md

26. user (2026-06-18T01:41:36.984Z)

scripts/README.md:7:- ? a CLI that orchestrates a pipeline/eval run end-to-end (e.g. `run_phase3_eval.py`, `run_process_and_stitch.py`)
README.md:133:`scripts/run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a hardcoded video path, reuses cached indexed segments if available (offering to re-index from scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`, `min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching settings against a fixed large recording without re-running expensive segmentation.
README.md:409: run_process_and_stitch.py # Interactive end-to-end runner
docs/CODE_MAP.md:29:| **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app code) | **`scripts/`** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
docs/CODE_MAP.md:119:- `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
docs/CODE_MAP.md:190:- `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw, segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time` (mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe fails (0.0) NO boundary validation; `begin_padding`/`end_padding` ctor defaults 0.5/1.0 but every live call site (`/api/compile`, CLI trim, `scripts/run_process_and_stitch.py`) threads `config.json stitching.{begin_padding,end_padding}` (typed default 0.5/0.5); temp clips in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses

`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so final concat can stream-copy ? change a preset and concat may break. ? optional **branding watermark** (`stitching.watermark`, default-on): `::_watermark_filter` builds a `-vf` (drawtext via `textfile=` + optional logo `movie/overlay`) applied to the per-clip re-encode only (concat stays `-c copy`); a failed watermark encode retries the clip WITHOUT it (never nukes the reel, and logs the ffmpeg stderr reason). Pass `watermark=` to `VideoCompiler(...)` (a `WatermarkConfig` model or dict; `None` = off). Default font = bundled Apache-2.0 `::_BUNDLED_FONT` (`backend/assets/fonts/RobotoSlab-Regular.ttf`) so it renders without fontconfig; `font_path` overrides.
docs/CODE_MAP.md:316:| Filter highlights by shot count | `config.json stitching.min_shot_count` ? enforced on BOTH `@backend/main.py::compile_highlights` (`/api/compile`) and `@scripts/run_process_and_stitch.py` (segments without `shot_count` metadata are exempt) |
docs/CODE_MAP.md:327:| Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or `@scripts/run_process_and_stitch.py::main` (edit hardcoded paths) |
docs/CODE_MAP.md:383:| (no dedicated test) | individual `pipeline/validators/*` impls, `main.py`, `scripts/run_process_and_stitch.py`, `pipeline/segmenters/{wasb_hybrid,motion}`, `eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly |

27. assistant (2026-06-18T01:41:50.214Z)

This is a real, documented entrypoint ? "Interactive end-to-end runner" ? used for iterating on stitching settings. The CODE_MAP explicitly notes it "skips validation" and reads the segmenter from config. So if the owner edits `config.json` to `default_segmenter: "motion"` (the script is meant to be run by editing hardcoded paths/config), the bug triggers on a non-cached run.

Let me verify the assessment dimensions:

Real? Yes. `len(Success/Failure)` raises `TypeError` (empirically confirmed), and `if not segments:` is dead code for `Result` returns. The base class contract (`Result[List...]`) and `backend/main.py`'s defensive unwrapping both confirm the script is the outlier that fails to unwrap.

Reachable? Yes, but conditionally:
- Requires `default_segmenter: "motion"` (not the shipped `gemini`, not the code default `yolo_hybrid` ? both return bare lists).
- Requires a non-cached video so the segment branch executes.
- Requires cv2/GPU (owner's machine, not this container).

Already guarded/tested? No. CODE_MAP line 383 confirms `scripts/run_process_and_stitch.py` and `motion` have "no dedicated test" ? exercised only indirectly. No guard against the Result type.

Intended behavior? No ? this is a genuine inconsistency. The script was written for segmenters returning bare lists (matching gemini/yolo_hybrid), and motion.py was migrated to the Result contract while this CLI was not updated, unlike `backend/main.py`.

Now I'll weigh severity/precision honestly. The trigger requires a deliberate, non-default config change (`motion`), and `motion` is itself a fallback/legacy CV segmenter that produces random fake metadata (shot_count via `random.randint`). The default path (gemini/yolo_hybrid) is unaffected. So it's real but low-severity and narrowly reachable ? which matches the candidate's own "low" severity and 0.7 confidence.

This is a genuine defect that a maintainer would acknowledge: the CLI uses `len()`/truthiness on an un-unwrapped `Result` that the motion segmenter returns. I'll affirm the finding.

28. user (2026-06-18T01:42:06.146Z)

Structured output provided successfully

29. assistant (2026-06-18T01:42:13.113Z)

Confirmed the candidate as a real, low-severity, reachability-bounded defect.

Summary: At `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-

base/scripts/run_process_and_stitch.py:81-84`, the script calls `len(segments)` and `if not segments:` on an un-unwrapped `Result`. The motion segmenter (`backend/pipeline/segmenters/motion.py:31,224`) is the only segmenter that returns `Success`/`Failure` rather than a bare list ? I empirically confirmed `len(Success(...))` raises `TypeError` and that `Success`/`Failure` are always truthy (defeating the "No play segments detected" guard). The trigger is selecting `default_segmenter: "motion"` (a valid, registered, but non-default value) on a non-cached run. The shipped default (`gemini`) and code default (`yolo_hybrid`) return bare lists and are unaffected, so it's latent for defaults. `backend/main.py` (lines 201/213, 838/843) does the correct `isinstance(result, Failure)` + `.value` unwrap that this CLI lacks; CODE_MAP confirms neither the script nor `motion` has a dedicated test.